

Architecture du projet Antigone RH

Analyse technique — backend, frontends, architecture logique/physique et question des microservices — 12 août 2026

1. Vue d'ensemble du projet

Antigone RH est composé d'un **backend unique** (Spring Boot / Java) et de **trois applications frontend** distinctes (React / Vite), chacune dédiée à un domaine métier :

Composant	Rôle	Volume approximatif
Backend/	API REST unique, logique métier, persistance	~22 800 lignes Java · 35 controllers · 36 services · 48 entités JPA · 99 DTO
frontend-rh/	Gestion RH (employés, congés, pointage, paie, documents)	100 fichiers ts/tsx
frontend-finance/	Facturation, dettes, charges, CNSS, TVA	33 fichiers ts/tsx
frontend-projects/	Projets, media plans, équipes, tâches, réunions	103 fichiers ts/tsx

Les trois frontends consomment **la même API backend** et partagent des entités transverses comme `Client` , `Employe` et `Compte` .

2. Faut-il une architecture en microservices ?

Verdict : non, pas dans l'état actuel du projet.

Constats

- **Échelle modeste** — ~23 000 lignes de code backend, taille cohérente avec une équipe réduite ; pas la volumétrie qui justifie une décomposition en services indépendants.
- **Couplage de données fort entre domaines** — `Client` et `Employe` sont référencés par clé étrangère à la fois par le module Finance (facturation) et le module Projets (media plans, équipes). Séparer ces domaines en services isolés imposerait soit de la duplication de données, soit des appels réseau synchrones permanents, soit des transactions distribuées.
- **Une seule base PostgreSQL** partagée par tous les modules — aucune séparation de schéma par domaine à ce jour.
- **Pas de besoin de scaling différencié** — rien n'indique qu'un module (Finance, RH ou Projets) doive être mis à l'échelle indépendamment des autres.
- **Déploiement déjà découplé côté frontend** — les 3 SPA sont buildées et déployées indépendamment (GitHub Pages), ce qui donne une partie des bénéfices d'isolation sans la complexité microservices côté backend.

Coût d'une architecture microservices ici

- Orchestration et service discovery à mettre en place
- Observabilité distribuée (traces, logs corrélés) à construire
- Gestion de la cohérence des données entre services (sagas, événements)
- Latence réseau ajoutée pour des opérations aujourd'hui en mémoire (jointures JPA directes)

Pour la taille et la maturité actuelles du projet, ce coût dépasse largement le bénéfice.

Recommandation

Rester sur un **monolithe modulaire** : conserver un seul backend, mais organiser progressivement le code par domaine métier (RH / Finance / Projets) plutôt que uniquement par couche technique. Cela prépare une éventuelle extraction future sans payer le coût opérationnel des microservices dès maintenant. Un passage aux microservices ne deviendrait pertinent que si plusieurs équipes distinctes travaillaient en parallèle sur des modules séparés, ou si un module précis avait des contraintes de scaling/disponibilité radicalement différentes des autres.

3. Architecture logique

Backend — architecture en couches (Spring Boot)

Controller → réception des requêtes HTTP, mapping DTO
Service → logique métier
Repository → accès données (Spring Data JPA)
Entity → modèle persistant (JPA / Hibernate)

Couches transverses identifiées :

Package	Rôle
dto/ + converter/	Découplage entité ↔ objet exposé à l'API (ex. <code>ElementsSalaireConverter</code> , <code>TranchesIrppConverter</code> pour les structures de paie/IRPP)
security/	<code>JwtAuthenticationFilter</code> + <code>JwtService</code> — authentification stateless par JWT, aucune session serveur
exception/	<code>GlobalExceptionHandler</code> — gestion centralisée des erreurs
scheduler/	4 jobs planifiés (<code>@Scheduled</code>) : alertes, archivage de tâches, deadlines, synchro Google Drive — traitement asynchrone in-process, sans broker externe

Autorisation (RBAC) : modèle `Role` + `Permission`, contrôlé au niveau des routes dans `SecurityConfig` (ex. `hasAnyAuthority("ROLE_ADMIN", "VIEW_FINANCE")`).

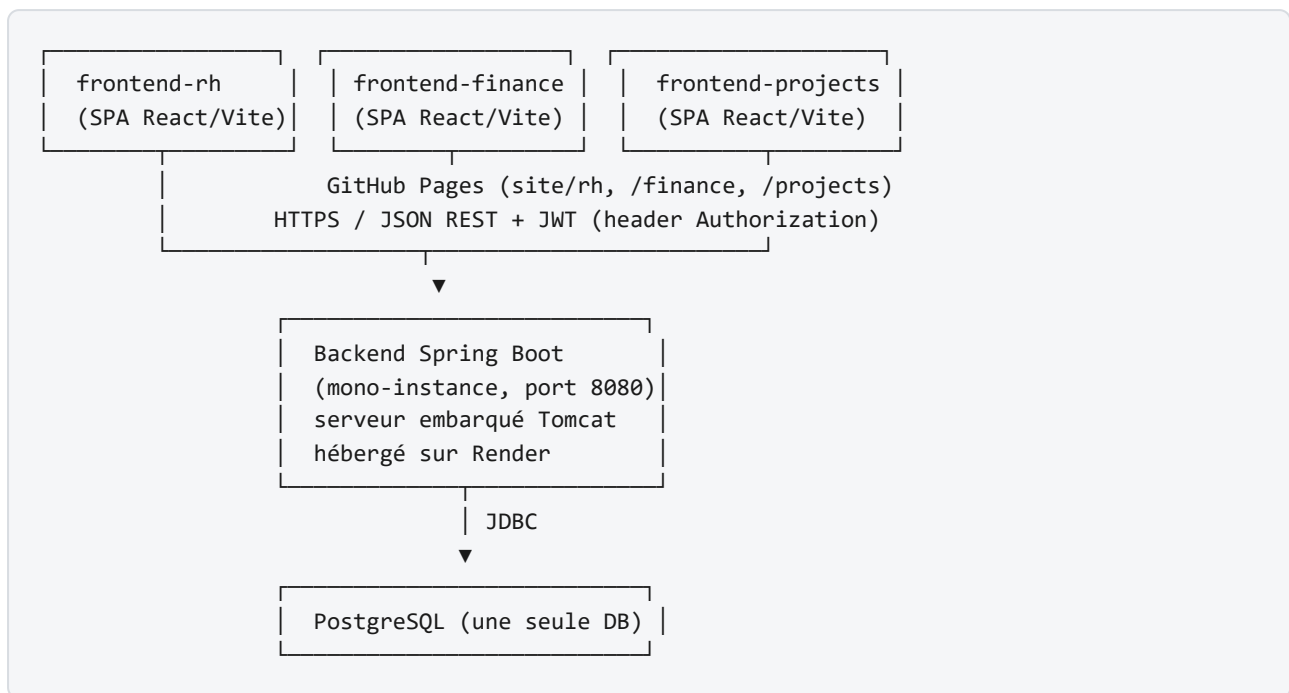
Le code est organisé **par couche technique** (`controller/`, `service/`, `entity/` ...) plutôt que par domaine métier — un monolithe classique, pas encore un monolithe modulaire au sens strict.

Frontend — architecture en couches (×3 SPA)

pages/ → écrans
components/ → UI réutilisable
context/ → état global (AuthContext, NotificationContext – React Context API)
api/ → services par ressource (axios)
hooks/, utils/, types/

`frontend-rh` et `frontend-projects` partagent une forte proportion de code (mêmes ~28 services API), signe d'une proximité fonctionnelle RH/Projets, tandis que `frontend-finance` est plus autonome (11 services propres : facturation, dettes, charges, CNSS).

4. Architecture physique (déploiement)



Intégrations externes (appels synchrones depuis le backend)

- **Google Drive API** — stockage des documents et médias
- **SMTP Gmail + Brevo API** — envoi d'emails
- **Groq API** — chatbot IA

Points clés

- **Une seule instance backend** exposant toute l'API REST — pas de gateway ni de load balancer dans le repo.
- **CORS piloté par configuration** (`app.frontend-url`) pour autoriser les 3 origines frontend — seul mécanisme séparant réellement les 3 apps ; le reste (auth, données, logique) est centralisé.
- **Stockage fichiers hybride** : uploads locaux (`uploads/justificatifs`) + Google Drive pour les médias.
- **CI/CD différencié par frontend** : le pipeline GitHub Actions ne rebuild que le(s) frontend(s) modifié(s) (filtrage par `paths:`) ; le backend est déployé séparément sur Render.

5. Serveur embarqué (Spring Boot)

Le backend utilise **Apache Tomcat** comme serveur embarqué, en version **10.1.53** (déclarée via `<tomcat.version>` dans le `pom.xml`).

C'est le comportement par défaut de `spring-boot-starter-web` : Tomcat est embarqué automatiquement, sans exclusion ni remplacement par Jetty ou Undertow dans les dépendances Maven. Tomcat écoute sur le port `8080` (configurable via `server.port` / variable d'environnement `PORT` sur Render).

Remarque : la dépendance `google-oauth-client-jetty` présente dans le `pom.xml` n'a aucun rapport avec le serveur web de l'application — elle est utilisée uniquement pour le flux OAuth local de Google Drive.